The Cost of Session Continuation in Multi-Agent Code Generation: A Measurement Study of a Commercial Coding-Agent CLI

Fudo

5 September 2026 Revision v0.3

Abstract

Continuing related coding tasks in one session can reduce repeated exploration while carrying history and adding scheduling constraints. We characterize a commercial coding-agent CLI and compare fresh sessions (B) with write-scope-connected lane continuation (C) on seven specifications in one Python repository (196 runs; 98 pairs). The median paired relative cost difference is +5.51% (BCa 95% interval: $[-3.18, +10.12]$%), failing the pre-specified 20% reduction criterion. Runtime and acceptance meet non-inferiority criteria with margins of 15% and 10 percentage points, respectively. Post-hoc analysis finds that two structural controls with no continuation also split in sign, cautioning against reading the small positive aggregate as an effect of continuation in general. The CT specification is costlier in all 14 pairs, with a median difference of +50.6%; outside CT, C is cheaper in 44/84 pairs, and the aggregate mean increase is concentrated in CT. Within CT, the two continued tasks take 21–30% fewer turns while their peak context grows by 46% and 91%, and the leading task—fresh in both arms—is costlier in all 14 replications (median $+42.8$%). Auxiliary-model use separates without exception: it is recorded in all 434 fresh calls and in none of the 84 resumed ones. Additional measurements show its cost scales with the submitted prompt, and per-call prompt sizes were not stored, so the size of its contribution to the main experiment is not determined. Fixed-rate accounting assigns the largest standardized increase to cache writes, but a residual relative to CLI-reported cost prevents a complete billing decomposition or attribution to inherited history alone. Excluding two structural unity predictions, four of five model predictions are within 25% relative error and all five overpredict. Beyond reporting the absent saving, the study describes several routes by which continuation changes cost, one of which—the auxiliary-model delegation stopping—was absent from the cost model. A single CT specification and a bundled intervention limit causal interpretation and generalization.

Keywords: coding agents, session continuation, prompt caching, cost measurement, pre-specified analysis

Version and status of the evidence. This is revision v0.3, dated 5 September 2026. The implementation corrections made to H1 and H3 in v0.2, together with the superseded results, are collected in Appendix A. The structural controls, sign tests, model evaluation restricted to continued specifications, and cost decomposition introduced in v0.3 are analyses added after seeing the results. They do not

replace the pre-specified primary decisions; the earlier manuscript, the raw data and the frozen predictions are all retained. We write “pre-specified” rather than “pre-registered” because we cannot demonstrate an independent public registration timestamp. Analysis procedures and artifacts are given in Appendices C and D.

Contents

1 Introduction

When several AI agents share the work of writing code, each of them may re-derive the same facts about a repository: its file layout, its functions, its implementation conventions. The human analogue is briefing a new colleague on the same background over and over. Asking the same colleague to continue would save the briefing; with language models, however, the earlier conversation and its tool output become part of the next input, and processing that input costs money. Caching lowers the unit price of those tokens but does not reduce the amount of context that must be processed to zero.

Our question is therefore not whether a conversation can be continued, but whether the exploration a continuation saves outweighs the cost of carrying its history. The subject is a commercial coding-agent CLI whose inference server and KV cache we cannot modify; what a user can choose is the execution order and whether to continue a session. A KV cache is the mechanism by which a model reuses the intermediate state of an already-processed input. We do not observe that internal state directly; what we obtain are the read and write token counts and the cost that the CLI reports.

Prior work covers the cost evaluation of caching strategies [7], the preservation and compaction of context [11], and improvements in time, quality and cost from task partitioning [14]. Building on that work, this paper combines a measurement of the continuation boundary in one specific CLI, a check of a cost model against measurements, and a comparison of one continuation policy driven by overlapping edit scopes. We do not claim that cache measurement itself, or the idea of grouping related tasks, is new.

Our contributions are threefold.

1. We separate what client-side records can and cannot establish about history reads on continuation, the conditions under which fresh sessions share cached input, fixed context, and the first-turn rewrite. The clearest instance is that the record of auxiliary-model use corresponds without exception to whether a session was reused — a route that our cost model did not contain.
2. We decompose the saving in re-exploration and the cost of carrying history, and we check a prediction frozen from independent arm-B calibration data against the main experiment.
3. We compare fresh sessions (B) against lane continuation (C), reporting the variation visible in structural negative controls, the increase concentrated in the CT specification, and a decomposition of cost by billing item and by task. Post-hoc analyses are kept distinct from the pre-specified comparison.

We neither build nor evaluate an adaptive scheduler or an optimal rule for switching conversations. Proposing candidate improvements from an observation is not the same as demonstrating them.

2 Background and related work

2.1 Caching and context reuse in commercial APIs

Lumer et al. [7] compare, across three commercial APIs, strategies that keep the full history, keep only the system prompt, or exclude dynamic tool results, evaluating cost and time-to-first-token on DeepResearch Bench. We differ in that we do not evaluate caching in general, but the fresh/continued boundary of a coding CLI and an execution policy that groups several tasks.

Santoni's Contextual Memory Virtualisation (CMV) [11] manages conversational state as a directed acyclic graph supporting checkpointing, branching and compaction, with a case evaluation over 76 coding sessions that also discusses the cost of reconstructing context. That observation motivates our question, but CMV's compaction and branching are a different intervention from lane continuation. We do not read our B/C comparison as direct evidence against CMV.

2.2 Dividing work across several agents

Co-Coder [14] partitions and schedules a dependency graph, addressing the relationship between inter-agent context transfer and parallelism. It evaluates pass rate, time and API cost on 28 problems and reports cost reductions. The claim that "existing coordination work does not measure cost" therefore does not hold. We do not optimize the partitioning; we build lanes from the edit scopes that existing tasks declare, and we measure that continuation policy together with the behaviour of the CLI.

2.3 Cache affinity and load balancing: a different layer of control

SGLang [16] builds KV reuse into the execution substrate through RadixAttention, and Parrot [5] conveys inter-request relationships to the serving side. Choosing which executor a request is sent to then places cache locality in tension with load balancing. DualMap [15] addresses that tension by mapping a prefix to two candidates and selecting by load. In practice, Ray's `PrefixCacheAffinityRouter` [10] prefers balancing when load is skewed, and llm-d [6] combines a prefix-locality score derived from KV events with a load score.

Section 6.1 of AgentServeSim [9] compares, in simulation, pinning the later turns of the same program to the same inference engine. On the two-instance configuration studied, pinned routing achieved a higher hit rate and a shorter p95 completion time than round-robin or least-loaded. That is a related finding in favour of affinity, but both the intervention and the measured quantity differ from ours. Sending the same conversation to a different engine is not the same comparison as merging several development tasks into one conversation, which changes both the history and the set of admissible execution orders. The former evaluates the benefit of avoiding recomputation; the latter evaluates a user's cost including changes in the amount of context. A reduction in time at the serving layer and the increase in USD cost we observe for CT are therefore compatible, and are not the same effect with opposite sign.

2.4 Practitioner reports and measurement tools

Issue 42338 of Claude Code [12] contains a user report, with token records, of increased cache creation immediately after resuming. ccusage [2] exposes per-session input, output, cache write/read and cost, and distinguishes stored cost values from recomputation against a rate card [1]. Together these show that the cost of resuming a session is a practical concern and that means of observing it already exist. We do not conclude from a report in one environment that "old sessions are always the most expensive" or that this is "widely believed". We take such reports as motivation and measure cost, including acceptance outcomes, under fixed task specifications and a fixed comparison design.

2.5 Pre-specification and the position of this work

Agent experiments admit many choices — of model, prompt, evaluation metric, and of changes made after seeing results — which has prompted arguments for pre-registration [13]. We stored our decision rules and amendment history before the experiment. We could not, however, obtain a published independent timestamp, and there were operational changes during the experiment as well as implementation corrections after analysis. We therefore state explicitly which parts were specified in advance and which were added or corrected later. Our contribution lies in the record of measurement and comparison; we do not present the complexity of a new optimization problem or an optimal algorithm for it.

3 Measuring cache behaviour, and a cost model

3.1 Units and terminology

A session is the unit of conversation that can be resumed while carrying its history. A task is one unit of development work; a run executes one specification under one policy and evaluates acceptance. Within a single CLI invocation for a task, several exchanges between the model and its tools (turns) may occur. We call a short behavioural check a probe and keep probes distinct from the 196 runs of the main experiment.

We call a state in which earlier input is available from cache warm, and a state in which that read is lost cold. There is also an intermediate state in which only fixed context survives. Classifications recorded as `ambiguous` by the read-ratio and write-ratio rule are not silently reinterpreted as warm or cold. The probes mostly used CLI 2.1.220 and the main experiment used 2.1.247; we do not assume that probe behaviour held identically across every call of the main experiment.

3.2 What we observed, and its limits

In the three M1 replications, inherited histories of 45,124, 45,123 and 45,122 tokens were reported as the same number of cache reads. The resume call cost 0.01438 USD, a median cost ratio of about $1/11.1$ against the immediately preceding initial

Measurement	Observation	Scope of interpretation
M1: warm resume	Three replications with a unique input prefix; history read ratio 1.0000 throughout	Confirms a cached read of the history under a small additional request. Does not imply a saving over the whole job.
M4: fresh to fresh	Three replications agree on the cache write/read split of the input	Under this condition we did not observe an additional discount from shared user input. We do not claim non-sharing under all conditions.
Fixed context, cross sharing	About 21.6k cache read even for a fresh request; sharing observed for a later fresh request after a continuation	Separates sharing of fixed context from conditional sharing of user history.
M6: growing history	Two series; the cache write of later turns stays close to the newly added amount	The view that a growing inherited history necessarily increases every write is not supported.
M7: added amount	A sweep from 500 to 16,000 characters and six replications at 2,000	A rewrite is not explained by a monotone threshold on the added amount. The association with the initial cache read is observational.
M2, M3: gap and switch	Two long-gap and two model-switch cases, both classified ambiguous	We establish neither an exact lifetime nor total loss on a model switch.

Table 1: Summary of the mechanism measurements. Detailed logs and the unit of classification are in Appendix B; k denotes thousands of tokens.

call. This compares calls inside a short probe; it is not the B/C comparison of solving the same coding problem.

In M7 we obtained a case where a 500-character addition triggered a rewrite and a 16,000-character addition did not. The rewrite coincided with observations of a small initial cache read, but we did not manipulate the provider's cache state independently. We therefore do not say that we identified the state as the cause; we say that the added amount alone does not explain the rewrite and that it corresponded to an indicator of the initial state. Reliable control in advance is not established, yet this does not leave the user with no information at all: adaptation based on

observation after the fact remains a candidate.

3.3 An explanatory decomposition and a break-even condition

Let e_j be the cost of re-acquiring context when task j starts fresh, H_i the history inherited from a preceding task i, and K'_j the number of turns after continuation. Let p_{in} be the base input price, α_r and α_w the price coefficients for cache reads and writes, and $\rho \in \{0, 1\}$ an indicator for a rewrite of the history on the first turn. In a simplified model the cost of carrying history is

$$c_{\text{carry}}(i, j) \simeq p_{\text{in}}\{\rho\alpha_w + (K'_j - \rho)\alpha_r\}H_i \tag{1}$$

If the cost of the actual work and of the fixed context can be approximated as equal across policies, continuation pays when

$$c_{\text{carry}}(i, j) < e_j \quad \Longleftrightarrow \quad H_i < H^*_j = \frac{e_j}{p_{\text{in}}\{\rho\alpha_w + (K'_j - \rho)\alpha_r\}}. \tag{2}$$

Here e_j is in USD, H_i in tokens and p_{in} in USD per token. The expression states that even with a cache, processing a long history over many turns increases cost.

This is not a finished decision rule. Continuation can also change the number of turns, the output, and the order of re-exploration, so cancelling the common costs is only an approximation. Nor can the rewrite indicator be determined reliably from the size of the added prompt or a nominal retention time. A user can vary the interval before resuming, but cannot directly control the overall cache state.

3.4 How the frozen prediction is actually computed

For model checking we used calibration data of one replication per specification, run under arm B only. We define the exploration segment as the turns before the first appearance of an editing tool (Write, Edit, NotebookEdit, MultiEdit) and the work segment as the turns after it. Writing $\widehat{S}_j$ for the minimum context observed in a task, we sum the context above it over exploration turns t:

$$\widehat{E}_j = \sum_{t \in \mathcal{E}_j} \max(0, \text{context}_{jt} - \widehat{S}_j) \tag{3}$$

This is a proxy built from tool names and token counts, not a direct measurement of what exploration means.

The frozen implementation `analysis/calibrate.py` takes arm-B measured cost as its base and predicts, for the second and later tasks in a lane,

$$\widehat{K}'_j = \max(1, K_j - K_{E,j}), \tag{4}$$

$$\widehat{C}_{C,j} = C_{B,j} - p_{\text{in}}\alpha_r\widehat{E}_j + p_{\text{in}}\{\rho\alpha_w + (\widehat{K}'_j - \rho)\alpha_r\}\widehat{H}_i \tag{5}$$

with ρ fixed at 1 and $\widehat{H}_i$ taken as the peak context of the preceding task in the arm-B calibration. The head of a lane takes $\widehat{C}_{C,j} = C_{B,j}$, and summing within a specification gives $\widehat{r}_s = \sum_j \widehat{C}_{C,j} / \sum_j C_{B,j}$. We keep the explanatory expression distinct from the implemented approximation, and do not claim that the implementation simulates the accumulation of history step by step.

The prices in the calibration implementation, for the recorded `claude-sonnet-5`, are 3.00 for input, 15.00 for output, 0.30 for cache read and 3.75 for cache write, per million tokens. These are fixed constants of that prediction code, not a statement about current service pricing. The base is arm-B measured cost including output charges, but the prediction does not explicitly model changes in output under continuation or a separate cache-creation charge. The prediction file is stored in commit `4fbb713`, made before the main experiment, and the corrections in this paper did not recalibrate it.

4 The execution policies compared

4.1 Shared constraints and the definition of a lane

This study ran two arms, B and C. The names begin at B because the labels of A, C′ and D are preserved as they stood when the protocol was frozen, with those arms out of scope; the gap is not the trace of an arm dropped after seeing results. What each arm was, and when and why it was set aside, is recorded in Appendix A.2.

Both policies respect precedence between tasks and never run two tasks concurrently when their declared edit targets overlap directly. Policy B checks that direct overlap against the running tasks and invokes each task in a fresh session. Policy C adds to the same shared constraint the rule that a lane is a connected component of the undirected graph whose edges join tasks with overlapping edit scopes — that is, a group linked directly or through other tasks. Only one task per lane runs at a time, and the second and later tasks are resumed with the previous session identifier. Parallelism between different lanes is retained.

4.2 Direct conflicts and transitive serialization

For three tasks A, B, C with no dependencies, whose edit scopes satisfy

$$W_A \cap W_B \neq \varnothing, \qquad W_B \cap W_C \neq \varnothing, \qquad W_A \cap W_C = \varnothing \tag{6}$$

policy B may run A and C at the same time, whereas under policy C all three belong to one connected component and run in sequence (Figure 1). That directly overlapping tasks cannot run concurrently does not entail that serializing an entire connected component carries no additional cost.

This comparison changes conversation continuation and in-lane serialization at the same time; it is a bundled intervention. The cost difference of policy C therefore cannot be separated into the causal effect of serialization alone or of caching alone.

Separating them requires a 2×2 design, and this study holds only the two cells on one diagonal.

The upper-right cell cannot be constructed. Under policy B, non-overlapping tasks run concurrently, and concurrent tasks cannot share one session; an operation that varies continuation alone is undefinable without presupposing serialization. Because of this asymmetry the single missing cell is the lower-left one, and we did not evaluate an additional arm to fill it. The head task of policy C is, however, in-

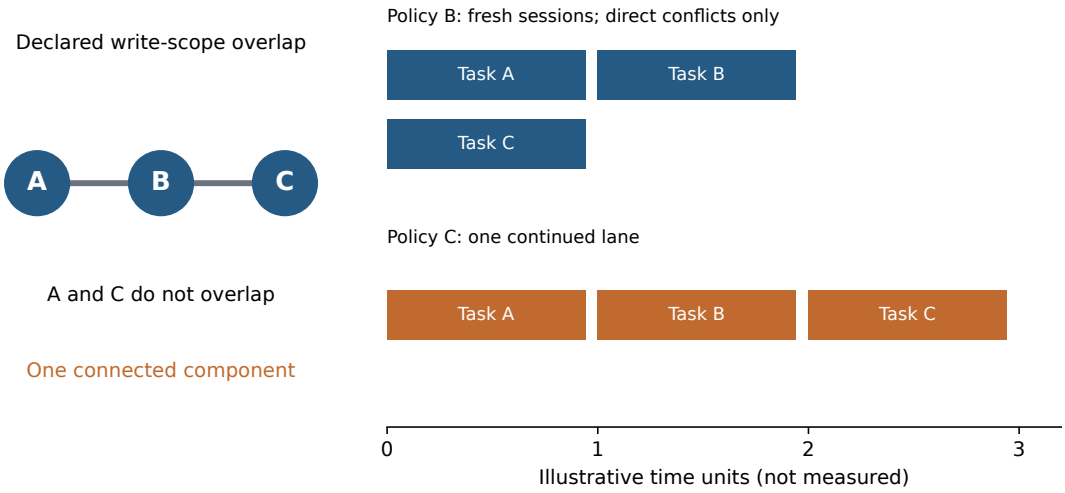

Figure 1: Schematic of the CT structure. Tasks A–B and B–C have overlapping edit scopes; A–C do not. Bar lengths are schematic and are not measured durations. Task names A/B/C are distinct from policy names B/C.

Scope of serialization	Fresh session	Continued
Direct conflicts only	Policy B	Not definable
Whole connected component	Not run	Policy C

Table 2: The design space of arms. The lower-left cell — serialize by connected component but invoke every task in a fresh session — would separate the effect of serialization from the effect of continuation. The upper-right cell cannot be constructed in principle.

voked in a fresh session under both arms, so a condition close to that cell is partially observed; the value is reported in §6.7.

4.3 Failures, switches and interruptions

The framework contains logic that abandons session inheritance when the representative model does not match. We fixed the representative model, but did not fix the CLI's internal use of auxiliary models to zero. Inspecting the implementation also shows that an acquired session identifier is recorded before a task failure is decided, so we do not adopt the earlier manuscript's statement that a failure always discards the session. Dependants of a failed task are blocked, but we did not evaluate a guarantee that continuation from a failed context is prevented on every path. We also distinguish a run-level acceptance failure from a task-level exception during a run. Lane-length limits, adaptive splitting and a guaranteed cache lifetime after a long interruption are not claimed as implemented or evaluated results.

5 Implementation and experimental method

5.1 Subject, specifications and scale

The execution substrate is an existing multi-agent framework and the subject under test is the Python repository `square-media`. We did not compare seven independent repositories. The subject was fixed to three commits (`d4449a0`, `3be3676`, `6f5b695`) that have acceptance tests and incomplete implementations. Each run executes in a separate Git worktree and inherits no artifacts from another run.

Specification	Shape	Tasks per lane	Role
`w_two_files`	W	$1+1$	Control with nothing to continue
`n_db_chain`	N	2	Already serial through dependencies
`m_mixed`	M	$2+1+1$	Dependency chain mixed with independent work
`s17_w_two_files`	W	$1+1$	Control from the other series
`s17_n_db_chain`	N	2	Dependency chain from the other series
`s17_m_mixed`	M	$2+1$	Mixture from the other series
`ct_library`	CT	3	No dependencies; transitive overlap of edit scopes

Table 3: Composition of the seven specifications. Differences within W include the variation of independently executed runs, not only an effect of session reuse.

Each specification was replicated 14 times under both policies, giving 98 pairs and 196 runs. The number of replications follows the pilot and the pre-specified procedure. The main experiment ran from 28 August to 2 September 2026; the recorded model identifier is `claude-sonnet-5` and the CLI version is 2.1.247. We report the model name as the identifier appearing in those logs. The final dataset contains no missing pairs and no post-hoc outlier exclusion, although interruptions and re-runs occurred before it was complete.

5.2 Order, intervals, and suppressing cache sharing

`build_schedule` permutes the specification order with seed 20260801 and places B first in odd replications and C first in even ones. This is not a design that randomizes arm order independently within each pair. Reconstructing start times by subtracting `wall_s` from the recorded end times gives 49 pairs with B first and 49 with C first, and all 98 pairs match the planned leading policy.

Partway through the experiment we changed the order from running one policy across all specifications first to running both policies adjacently within each specification (amendment A-5). This addressed the situation in which a usage limit leaves only one side of a comparison; it did not change any decision threshold for cost. The interval between the end of the leading run and the start of the following run had a median of 0.27 seconds and a maximum of 38.24 minutes, with 11 pairs separated by more than ten minutes. Because these are estimated from end times and rounded durations, sub-second values are not precise measurements of waiting time.

Each run places a unique string (a nonce) in its prompt so that the user-input prefixes of different runs are unlikely to match, while keeping that string constant

within a run. This measure does not remove sharing of fixed system context or time-varying behaviour on the service side. We do not claim that balancing the order alone eliminates confounding by cache state, and we also report a sensitivity analysis that excludes widely separated pairs.

5.3 Records, and the definitions of cost and quality

For every CLI invocation, `calls.jsonl` stores the task identifier, the session identifier, whether the call was a continuation, the model, the CLI version, input, output and cache token counts, cost, and the number of turns. Turn-level information obtained from `--output-format stream-json --verbose` is also retained. Total cost is the provider-reported USD cost summed within a run; we did not reconcile it independently against an invoice. The dataset and the cost in `run.json` agree across all 196 runs. The accounting analysis uses this run-level aggregate and keeps it distinct from summing every line of an appended call history. The CLI's representative model identifier denotes the model with the largest cost, and auxiliary models are also used. The log stores the names in `models_used`, but not the per-model amounts and token breakdown from the original `modelUsage`. `DISABLE_NON_ESSENTIAL_MODEL_CALLS`, which suppresses auxiliary-model calls, was not set; the experiment measures the CLI's default behaviour.

Time is the elapsed time from submitting a run to completing its acceptance decision, not the model's inference time alone. Quality is binary: one if every acceptance test passed, zero otherwise. Mean pass rate over individual tests, code maintainability and human quality scoring are not part of this metric. Agreement between declared edit scope and actual writes, scope drift, and LLM-judge scoring were not measured.

5.4 Interruptions and the stopping condition

An earlier aggregation divided 121 infrastructure interruption events by 196 runs, obtained 61.7%, and thereby tripped a 20% stopping condition. Amendment A-6 records that collapsing repeatedly interrupted runs of the same identity gives 27/196 (13.8%), and the decision to resume was taken before the comparison values were examined. We retain both A-6's reasoning and the fact that the earlier aggregation detected a stop. An event ratio and a run ratio are different quantities, and neither is deleted without explanation. Operational interventions — usage limits, false positives in a local block, worktree inconsistencies, the order change — occurred, and their individual effects were not separately evaluated.

6 Evaluation and results

6.1 What is evaluated, and the pre-specified criteria

For pair i we write the relative cost difference as $d_i = (C_i - B_i)/B_i$; a negative value means C is cheaper. The primary point estimate is median(d_i) over all 98 pairs, not the median of per-specification medians. We interpret it as an aggregate over a fixed composition of seven specifications with equal replication.

Hypothesis	Question and estimand	Pre-specified criterion
H1 cost	Paired relative difference, median	Two-sided sign-flip test $p < 0.05$, BCa 95% interval excluding 0, and median $\leq -20\%$
H2 time	median(T_C)/median(T_B)	One-sided BCa upper bound < 1.15; the implementation uses 90%
H3 acceptance	$p_C - p_B$	One-sided 90% lower bound > -10 percentage points
H4b prediction	Per-specification total cost ratio r_s	Relative error within 25% for a majority, and agreement in direction for every specification

Table 4: Summary of the decision rules. The relative scale for H1 follows decision A of the pre-analysis document. The H2 and H3 margins are operational judgements.

The statistic of the sign-flip test is the mean, while the estimand for practical significance and for the interval is the median. We use 10,000 resamples with seed 20260802 and compute test p-values as $(b + 1)/(10{,}000 + 1)$ [8]. BCa is the bias-corrected and accelerated bootstrap interval [4]. Because the test and the interval address different statistics, a $p < 0.05$ alongside a median interval that spans zero is not a contradiction. The analysis assumes independence and sign exchangeability; we do not claim that randomization inference follows automatically from a deterministic alternating order.

6.2 The status of the auxiliary analyses

The structural controls, exact sign tests, model evaluation restricted to the five continued specifications, the CT cost decomposition, and the mechanism observations of §6.7 are post-hoc descriptive analyses. The frozen criteria of H1 through H4b are retained, and no additional analysis becomes a new primary decision. Besides all 98 pairs, we report four groupings: 84 non-CT pairs, 28 control pairs, 70 continued pairs, and 56 pairs excluding both. These are subsets of the same data and were not designed as multiple comparisons with an adjusted significance level. No subgroup is used in place of a pre-specified criterion. Post-hoc sections are marked in their headings below; the unmarked §6.3, §6.8 and §6.9 carry the pre-specified decisions. The cost decomposition uses the prices frozen in the §3 calibration and keeps the residual against CLI-reported cost as an independent item. Definitions of the recomputations and the scope of log reconciliation are in Appendix C.

6.3 Cost: the 20% reduction criterion was not met

The median is not in the direction of a reduction and the 95% interval spans zero, so the composite criterion of H1 is not met. The lower end of the interval lies above -20%, so under the assumptions of the analysis the aggregate median observed here is inconsistent with a reduction of 20% or more. This is not a conclusion that there is no effect at all, or that no specification can be made cheaper. Figure 2 shows values spread widely, with many comparisons in which C is cheaper.

C is cheaper in 44 of 98 pairs (44.9%), and a two-sided exact sign test using only

Quantity	Result
Median relative cost difference	$+5.51\%$
Sign-flip test on the mean relative difference	$p = 0.0013$
BCa 95% interval for the median	$[-3.18, +10.12]\%$
Pairs where C is cheaper	44/98
Total cost B / C	58.35 / 62.91 USD
Increase in total cost	$+7.82\%$

Table 5: Overall cost results. The ratio of totals and the median of paired ratios are different statistics.

the direction gives $p = 0.3634$. The mean relative difference, however, is $+8.37\%$, and a sign-flip test on the mean gives $p = 0.0013$. Frequency and magnitude are different quantities, and we do not reread the smaller p-value as strong evidence that "C is more expensive in many pairs". Overall, 54 costlier pairs outnumber 44 cheaper ones, so the positive median is consistent with that ordering; we do not explain it as the median being dragged by a few extreme increases.

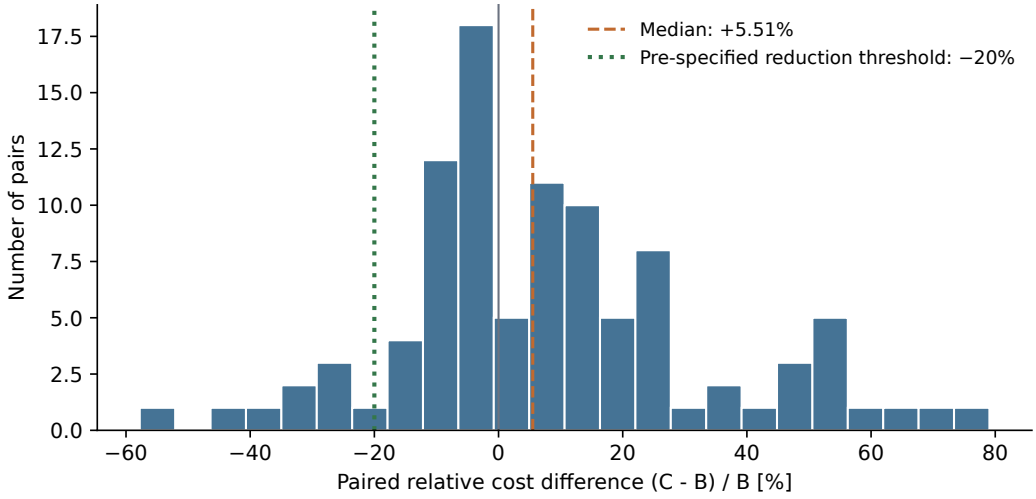

Figure 2: Relative cost difference over all 98 pairs. The dashed line is the median and the dotted line the 20% reduction criterion. Variation across comparisons is distinguished from the aggregate.

6.4 Per-specification and CT-excluded sensitivity (post-hoc)

This section may appear to point the other way from the previous one, but the statistics differ. A median is determined by ranks and is therefore sensitive to the composition of specifications; a mean is determined by magnitudes and is therefore sensitive to the large CT differences. Both are properties of the same 98 pairs, and neither makes the other wrong.

For `ct_library` the median relative difference is $+50.6\%$ and C is costlier in all 14 pairs. The other six specifications have medians between -7.9% and $+12.1\%$. The secondary aggregate over the 84 non-CT pairs gives a median of -1.4%, $p = 0.4950$ on the mean-based relative scale, and a BCa 95% interval of $[-5.10, +6.44]\%$. All 44 pairs

Spec	Median B	Median C	Median rel. diff.	C cheaper
`s17_w_two_files`	0.5547	0.5293	-7.9%	10/14
`n_db_chain`	0.4323	0.4467	-5.6%	10/14
`s17_n_db_chain`	0.4501	0.4790	-1.4%	8/14
`s17_m_mixed`	0.7451	0.7907	$+5.0\%$	5/14
`m_mixed`	0.8054	0.8501	$+5.2\%$	6/14
`w_two_files`	0.3609	0.3722	$+12.1\%$	5/14
`ct_library`	0.6617	1.0255	$+50.6\%$	0/14

Table 6: Per-specification cost (USD) and paired relative differences. The median relative difference cannot be recomputed from the medians of B and C. Fourteen pairs per specification.

in which C is cheaper lie in this non-CT group, a slight majority at 44/84 (52.4%; sign test $p = 0.7436$). Adding the 14 CT pairs raises the count of positive differences from 40 to 54 and flips the sign of the median. Decomposing the mean relative difference into additive contributions over all 98 pairs gives $+7.12$ points from CT and $+1.25$ points from the rest. CT accounts for 14 of 98 pairs yet contributes most of the overall mean of $+8.37\%$. This is a decomposition of the mean, not of the median of $+5.51\%$.

Excluding both the two W specifications and CT leaves 56 pairs — shapes that are not direct-conflict controls and in which continuation actually occurred. Their median is -0.65%, the BCa 95% interval is $[-4.77, +6.86]\%$, and C is cheaper in 29/56 (sign test $p = 0.8939$). The interval contains 0 and contains neither -20% nor $+20\%$. There is, however, no pre-specified equivalence margin on the cost side, so we do not read this as demonstrating cost neutrality. Descriptively, neither a large increase nor a large decrease was observed in this range.

We can say that the sign of the overall median is sensitive to the composition of specifications, but we do not substitute the CT-excluded value for the primary result. CT is a single specification, so the effect of the shape in general cannot be separated from the content of that particular task. Figure 3 shows every observation by specification.

6.5 Variation seen through structural negative controls (post-hoc)

In the W-shaped `w_two_files` and `s17_w_two_files` every connected component is a single task, so neither conversation continuation nor the additional serialization it would bring can occur. All 56 stored C-side calls are `resumed=false`. Because B and C coincide with respect to the scheduling and continuation operations under test, we read these 28 pairs as structural negative controls. The two runs were nevertheless executed separately and were not made identical down to stochastic generation, timing and nonces.

The median of `s17_w_two_files` is -7.9% and that of `w_two_files` is $+12.1\%$. The overall $+5.51\%$ lies between these two points, as do the medians of the other non-CT specifications. The only per-specification median above that range is CT's $+50.6\%$. That range is, however, the minimum and maximum of two point estimates, not an upper bound on measurement noise. Individual W pairs range from -43.4% to

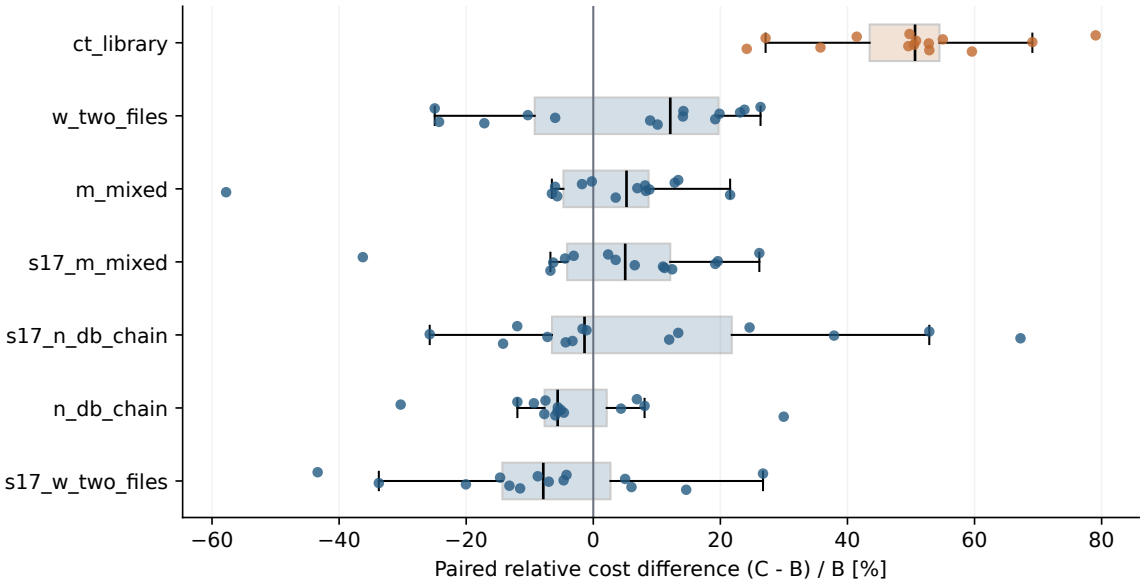

Figure 3: All observations and box plots by specification. The 14 CT pairs are all above zero; the other specifications mix positive and negative values.

Spec	n	Median rel. diff.	BCa 95% CI	Median USD diff.
`s17_w_two_files`	14	-7.9%	$[-14.67, +0.40]\%$	-0.0376
`w_two_files`	14	$+12.1\%$	$[-10.29, +19.53]\%$	$+0.0404$
`ct_library`	14	$+50.6\%$	$[+41.46, +53.95]\%$	$+0.3422$

Table 7: The two control specifications and CT. Intervals are post-hoc BCa intervals for each specification's median relative difference, not tests of differences between specifications.

$+26.7\%$ and overlap CT's minimum of $+24.2\%$. We do not derive from this comparison a verdict that "only CT exceeded noise statistically".

Pooling the 28 W pairs gives a median of -4.44%, a BCa 95% interval of $[-11.52, +10.12]\%$, and C cheaper in 15/28. There is no basis for correcting other specifications by a single control effect, and we apply no such subtraction. Each W specification has only 14 pairs and median B costs of 0.36 and 0.55 USD, so we cannot assume that cost scale and task-specific variation are shared with the other specifications. Figure 4 plots all observations with their intervals, separating a small aggregate difference from the consistently large increase observed for CT.

6.6 CT by billing item and by task (post-hoc)

Within CT, the reported total over 14 runs per arm rises from 9.5434 USD under B to 14.2279 USD under C, a difference of 4.6845 USD ($+49.09\%$). What follows decomposes that difference in totals; it is not a decomposition of the paired median of $+50.6\%$.

Writing T_k for the four token items, p_k for the frozen calibration prices, and ϵ for the residual against the reported amount,

$$c_{\text{reported}} = \sum_{k \in \{\text{in,out,read,write}\}} p_k T_k + \epsilon, \qquad \Delta c_{\text{reported}} = \sum_k p_k \Delta T_k + \Delta\epsilon. \tag{7}$$

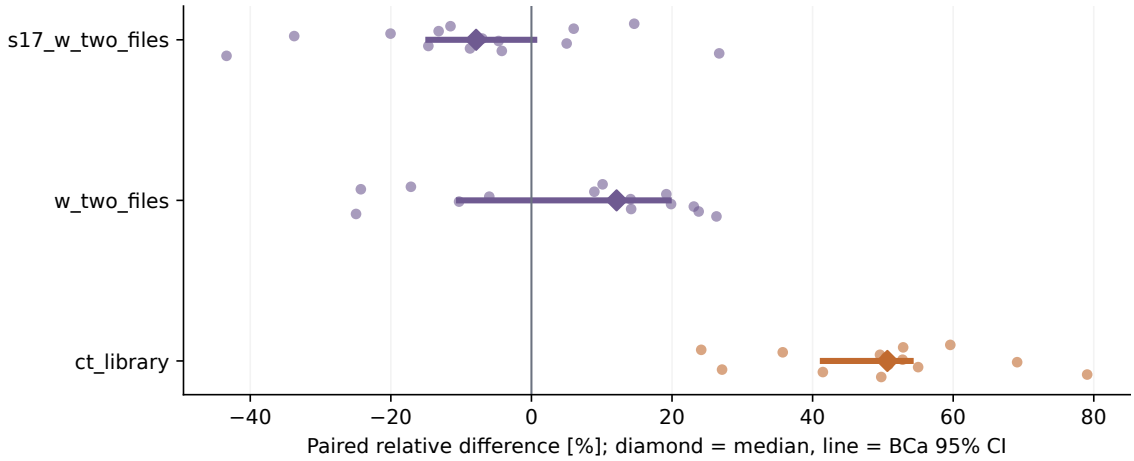

Figure 4: Controls and CT. Points are the 14 pairs, diamonds the medians, horizontal lines their BCa 95% intervals. The two control medians are not drawn as a background band or a significance threshold.

The prices are 3 for ordinary input, 15 for output, 0.30 for cache read and 3.75 for cache write, per million tokens. This standardizes by the calibration code's prices; it does not independently verify the per-model billing rates of this experiment. The stored `usage` and `total_cost_usd` are different aggregates and the per-model breakdown cannot be recovered, so we do not force the residual to zero. The residual ϵ absorbs the cost of the auxiliary model in full: across 27 stored probe records the four `usage` items match the main model's own figures exactly, so the auxiliary model's tokens never enter that aggregate while its cost is added to `total_cost_usd` (§6.7).

Item	Token delta	B	C	Delta
Input	-6	0.0032	0.0032	-0.0000
Output	$+5,527$	2.1744	2.2573	$+0.0829$
cache read	$+6,434,057$	5.9230	7.8532	$+1.9302$
cache write	$+917,288$	3.5554	6.9952	$+3.4398$
Residual vs reported	—	-2.1126	-2.8810	-0.7684
Reported cost	—	9.5434	14.2279	$+4.6845$

Table 8: Additive decomposition over the 14 CT run pairs, in USD. Columns B and C convert the four token items at fixed prices; only the residual row is not a converted amount but the balancing term against the reported cost. The four items and the residual sum to the CLI-reported amount. The ordinary-input difference is -0.000018 USD.

Of the 5.4529 USD increase at fixed prices, cache write accounts for 3.4398 USD (63.1%) and cache read for 1.9302 USD (35.4%). The change in the residual against reported cost is -0.7684 USD, which is not negligible rounding. These shares therefore apply to the increase converted at fixed prices, not to the composition of the actual billed increase. Cache read mixes fixed context, within-task history and inherited history, and token counts alone cannot identify their origin.

CT cache reads rose from 19,743,378 to 26,177,435 tokens ($+32.6$%), cache writes from 948,096 to 1,865,384 ($+96.8$%), and output from 144,960 to 150,487 ($+3.8$%). The CLI-reported turn total fell from 587 to 572 (-2.6%), so the overall cost increase

cannot be explained by more turns. Recorded turn detail rows number 551 and 530 and do not match the reported counts, and per-turn input aggregates also disagree in part, so we do not reconstruct a complete per-turn bill from them.

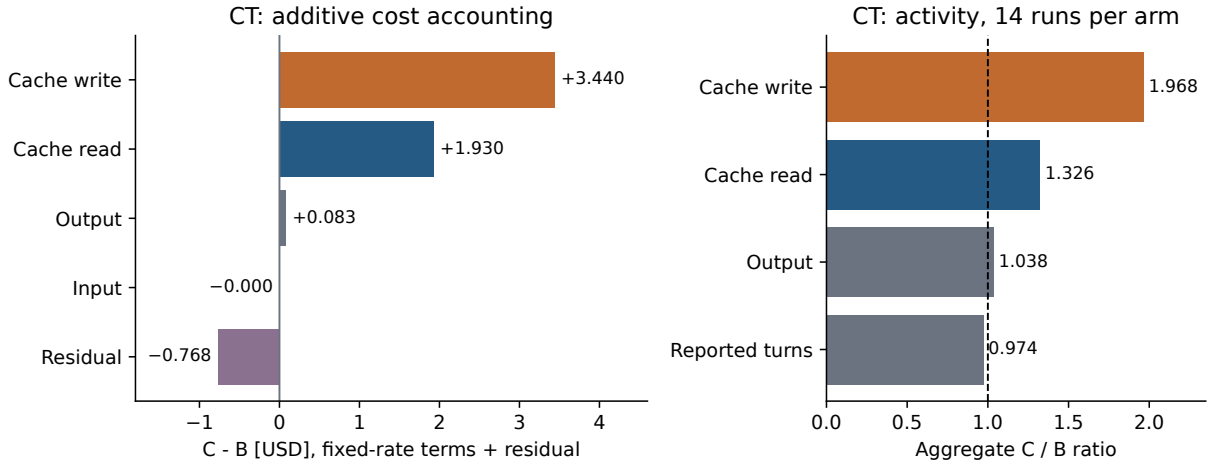

Figure 5: Fixed-price cost decomposition for CT (left) and ratios of usage and reported turns (right). The negative residual is shown, preserving the difference between converted and CLI-reported amounts.

Task	Cost B	Cost C	Delta	Turns B/C	Cont. in C
`CT-A`	3.5933	4.9741	+1.3808	238/316	no
`CT-B`	3.2771	4.5713	+1.2942	201/153	14/14
`CT-C`	2.6729	4.6824	+2.0095	148/103	14/14

Table 9: Reported cost per CT task (sum of 14 replications, USD) and CLI-reported turn counts. CT-A is a fresh session under C as well.

By task, 1.3808 USD — 29.5% of the total CT increase — appears in CT-A, the leading task, which starts fresh under C as well. CT-A's turns rise from 238 to 316, while the two continued successors fall from 349 to 256 in total. An account that attributes the whole CT increase to the direct cost of reading inherited history is therefore unsupported at the accounting stage as well. The leading task also runs in an environment whose ordering constraints have changed, and generation varies, so we do not assign this difference to a specific cause. Filling in H_i, ρ and K'_j of §3 from proxies is a model computation; these per-item observations do not amount to measuring the terms of the carry expression directly.

6.7 Mechanisms observed under continuation (post-hoc)

We describe which quantities change alongside the change in cost, for the three CT tasks. This too is post-hoc and does not replace the H1–H4b decisions.

First, session reuse and the composition of models used correspond exactly. All 434 calls in fresh sessions record use of an auxiliary model (`claude-haiku-4-5`), and none of the 84 calls in continued sessions do. There is no exception among the 518 call records, and the correspondence holds in every one of the seven specifications. It admits two readings: that continuation does not delegate to the auxiliary model,

or that the processing which uses the auxiliary model occurs only when a session starts. The first would be a cost of continuation, the second an incidental cost of starting fresh; the presence or absence of a recorded use cannot decide between them. Either way, comparing per-turn quantities between arms mixes the amount of history with a difference in model composition. The log does not store per-model amounts and token counts, so the two cannot be separated. Equation (1) does not contain this term. We report the per-turn cost ratio of Table 10 descriptively and do not read it as the cost of processing inherited history.

The main experiment discarded the per-model breakdown, but the probes did not. Their stored raw results retain `modelUsage`, from which the auxiliary model's cost and tokens can be read directly. Across 27 fresh-side records the auxiliary model consumed 12,043 input tokens and produced 15 output tokens, with zero cache read and zero cache write, at a cost of 0.012116 USD.

It is tempting to multiply that figure by the call counts of the main experiment, and it cannot be done. Measurements taken on 8 September 2026 under CLI 2.1.247, varying only the number of prefix lines, show the auxiliary input tracking the submitted prompt almost proportionally: 905 tokens for no prefix (an estimated 9 prompt tokens), 3,795 for 125 lines (3,300), 6,669 for 250 lines (6,581) and 23,918 for 1000 lines (26,268). The auxiliary model reads the prompt it is given. What made it look constant near 12,000 in the probes is that every probe used the same 500-line prefix, holding the prompt size fixed.

The per-call cost therefore depends on the prompt size of that call. The task descriptions of the main experiment have a median length of 393 characters, but the amount of context the execution framework adds was not stored, so per-call prompt sizes cannot be recovered. At 100 prompt tokens the auxiliary cost would be 0.0001 USD; at 12,000 it would be 0.013 USD, a factor of 123. The magnitude of the auxiliary model's contribution to the main experiment is therefore not determined. What the probes establish is that it exists, that it corresponds without exception to session reuse, and that its cost scales with the prompt.

The same additional measurements show that setting `DISABLE_NON_ESSENTIAL_MODEL_CALLS` to 1 does not suppress the call: it was recorded in all four runs with the variable set and all four without, with no difference in input or cost. Suppressing the auxiliary model would require some other means.

Task	Cont. in C	Median Δturns	Peak ctx B→C	Cost/turn	Median Δcost
`CT-A`	no	+5.5	43,290→52,637	1.04	+42.8%
`CT-B`	yes	−3.0	44,044→64,482	1.83	+37.0%
`CT-C`	yes	−3.0	38,408→73,340	2.52	+84.1%

Table 10: CT by task. Turn deltas and relative cost differences are medians of the paired differences over 14 replications; peak context is the median within each arm. The per-turn cost ratio includes the difference in model composition and is not a unit price for processing history.

For the two continued tasks, turns fall and output is nearly unchanged while peak context rises from 44,044 to 64,482 (+46%) and from 38,408 to 73,340 (+91%). Cost rising while a proxy for the amount of work falls is consistent in direction with what equation (1) describes. Peak context nevertheless includes fixed context and within-

task history, so we do not assign the increase to inherited history alone.

Cost also rose for CT-A, the leading task, which is not continued. The median paired difference over 14 replications is +42.8%, with a BCa 95% interval of $[+28.39, +49.17]$%; turns rose in 14/14 replications, and peak context and output rose as well. CT-A is a fresh session under both arms, so this difference cannot be explained by continuation itself. Moreover, an auxiliary model is used in all 14 calls of both arms for CT-A, so the caveat stated at the head of this section — that per-turn quantities mix model composition — does not apply to this one task. The per-turn cost ratio stays near 1.04 and the increase appears as more turns and more output, which means the amount of work itself grew rather than its unit price. What differs between arms is the ordering constraint, not the input given to the leading task: under policy B the edit scopes of CT-A and CT-C do not overlap and the two may run concurrently, whereas under policy C they join the same lane and do not.

In this sense CT-A can be read as an accidental observation, over 14 pairs, of the missing lower-left cell of Table 2: the condition in which serialization changes without continuation. It is not an experiment that fills the design gap, but the fact that the value in that direction was not small is worth reporting. We cannot explain why the presence or absence of concurrency changes the amount of work in the leading task itself. Provider-side state, sharing of the execution environment and framework behaviour are all candidates, and our logs cannot distinguish them. We present this as an unresolved observation and do not identify a cause.

For the inherited amount $\widehat{H}_i$ entering equation (1) we used the peak context of the preceding task in the arm-B calibration (§3.4). Against that, the context growth observed in the main experiment reached only 49% of the assumed amount for CT-B and 81% for CT-C. Observing less inheritance than assumed agrees in direction with the fact that all five continued specifications were overpredicted (§6.9). The direction agrees, but the magnitude does not. Substituting the observed ratios back into the frozen expression lowers the CT prediction from 1.5575 to 1.3597, and the relative error moves from -4.3% (over) to $+9.6$% (under): the error does not shrink but changes sign and more than doubles. The account that "an overly large assumed inheritance causes the overprediction" therefore does not hold quantitatively, at least for CT. It is more consistent to suppose that terms absent from the model act in the opposite direction and that the inflated inheritance assumption partly cancels them. The auxiliary-model delegation stopping, and the increase in work in the leading task, are both terms absent from equation (1). The same additional measurements show that setting `DISABLE_NON_ESSENTIAL_MODEL_CALLS` to 1 does not suppress the auxiliary call: it was recorded in all four runs with the variable set and all four without, with no difference in input or cost. That environment variable does not govern this call.

This substitution is a sensitivity check, not a recalibration; the frozen prediction of §6.9 is unchanged. Peak context is not a per-turn mean, and the calibration used one replication against the experiment's fourteen.

Finally, the ratio of reported cost to the fixed-price conversion falls between 0.78 and 0.85 across the fourteen specification-arm points (overall 0.804 for B and 0.820 for C). The residual acts as an approximately constant proportion rather than as additive error. That the cheaper auxiliary model is excluded from the conversion

is consistent with the ratio being below one. Furthermore, the arm that uses the auxiliary model more — B — should have the smaller ratio, and the measurements follow that direction in 5 of seven specifications and overall. The two control specifications, in which no continuation occurs, split in direction, which is consistent with no systematic difference being visible there. Two independently obtained descriptions point the same way, but the residual's per-item composition is not observable, so we do not treat this as identifying a mechanism. The two arms having similar ratios indicates that the item shares of §6.6 are not in a regime where the residual would rearrange them substantially. It is not a proof that the shares are preserved, and we do not refit the prices to the measured totals.

6.8 Time and acceptance tests

The median runtime is 280.85 seconds for B and 292.80 for C, a ratio of 1.0425. The one-sided 90% upper bound from a paired bootstrap, 1.1335, is below 1.15 and meets the specified non-inferiority criterion for time. This is not a proof that the times are equal; it is a judgement with respect to the 15% margin adopted.

Full acceptance was reached in 98/98 runs under B and 97/98 under C, a difference of -1.02 percentage points. The one-sided 90% BCa lower bound on the difference in pass rate is -5.10 points, meeting the -10 point criterion. That margin is comparatively wide, and data with a single discrepancy cannot guarantee equivalence of quality in general. As a supplementary check, inverting the one-sided 90% Clopper–Pearson upper bound on the probability of B succeeding while C fails gives a conservative lower bound on the pass-rate difference of -3.91 points (Appendix C).

6.9 Where the model prediction agrees and disagrees

The measured ratio for specification s is $r_s = \sum_i C_{si} / \sum_i B_{si}$, which differs from the median relative difference of Table 6. Evaluated over all seven pre-specified specifications, six of seven fall within 25% relative error but agreement in direction across every specification fails, so H4b does not hold. The predicted value of 1.0000 for the two W specifications, however, follows structurally from the absence of continuation. In a post-hoc auxiliary tabulation that separates those two, four of the five genuinely continued specifications fall within 25%, the exception being `n_db_chain` at -37.2%.

Spec	Observed r_s	Predicted $\widehat{r}_s$	Rel. error	Within 25%
`ct_library`	1.4909	1.5575	-4.3%	yes
`m_mixed`	1.0060	1.1806	-14.8%	yes
`n_db_chain`	0.9571	1.5251	-37.2%	no
`s17_m_mixed`	1.0205	1.2987	-21.4%	yes
`s17_n_db_chain`	1.0736	1.3730	-21.8%	yes
`s17_w_two_files`	0.9021	1.0000	-9.8%	yes
`w_two_files`	1.0334	1.0000	$+3.3$%	yes

Table 11: Comparison against the frozen prediction. Relative error is $(r_s - \widehat{r}_s)/\widehat{r}_s$. A prediction of exactly 1.0000 is not a prediction of an increase.

For the five continued specifications the prediction exceeded the measurement in every case. This describes the bias of the part of the model that handles continuation more directly than the 6/7 overprediction over all seven. Because those five points share the same model expression and calibration procedure, we do not treat them as independent fair sign trials and do not claim the pattern is "hard to explain by chance". The bias is a reason to re-examine the single-replication calibration, the fixed $\rho = 1$, the proxy for the exploration segment, the approximation of accumulated history, and the discrepancy between fixed prices and CLI-reported cost. We have not established a cause or a correction, and we did not recalibrate the prediction to the present data.

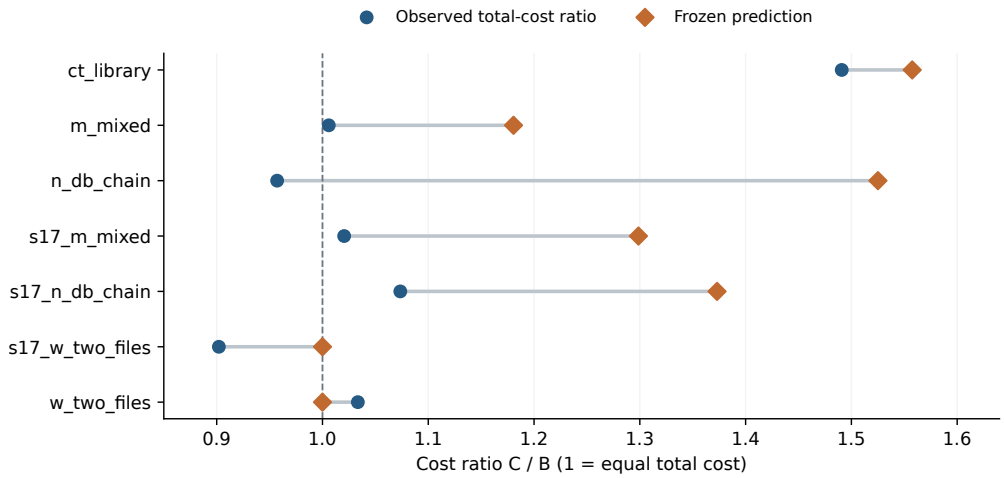

Figure 6: Prediction versus measurement. All five continued specifications are overpredicted. The lower two W points are controls that return 1.0000 structurally and are separated from the auxiliary tabulation. The dashed line at 1.0 marks equal total cost between policies.

6.10 The central claim, and additional analyses

The central claim — "reduce cost by at least 20% while keeping time and quality within tolerance" — holds only if every gate is met. The cost condition is not met, so the central claim is not supported. We report only the non-inferiority of time and acceptance, with their margins.

As an additional analysis during revision, we bootstrapped by resampling 14 pairs with replacement within each of the seven specifications. The percentile 95% interval of the pooled median was $[-2.17, +8.84]$%. Preserving the replication structure per specification, this analysis remains inconsistent with a 20% reduction. It is an additional analysis conditioned on the fixed seven specifications and is not an interval over a population of repositories. Excluding the 11 pairs separated by more than ten minutes leaves 87 pairs with a median of $+6.0$% and a BCa interval of $[-3.27, +8.95]$%, again failing the cost criterion.

7 Views that were not supported

This section lists explanations we decided not to adopt during measurement, and matters we do not count among our results. It advances no new claims; the mechanism observations are in §6.7.

7.1 Cache sharing is not a simple binary

We initially framed the question as whether sending the same input in a different session shares the cache or shares nothing at all. In fact fixed context is shared, and the observations for fresh-to-fresh differ from those for a fresh request after a continuation. We therefore do not adopt the general statement that "fresh sessions gain nothing from the cache". The comparison experiment accounts for that conditionality through nonces and recorded execution order.

7.2 Growing history and rewriting are not the same thing

In M6, even as the inherited history grew, the cache write of later turns stayed close to the newly added amount. A long history is not the same as rewriting all of it every time. M7 likewise did not support explaining the first-turn rewrite by the size of the added prompt alone. The cost of repeatedly processing history through cache reads nevertheless remains. Observing part of a mechanism does not let us estimate the net benefit over a whole job. These observations come from probes; how much the context of continued tasks actually grew in the main experiment is reported in §6.7.

7.3 Arms that were not run are not results

C′, which switches to a fresh session once the history exceeds a budget; A, a single-agent reference point; and D, an adaptive scheduler, were none of them executed. What was not run is neither supported nor refuted, so we claim nothing here. When and why each was set aside is recorded in Appendix A.2. We do not say that a small number of observations identified an exact break-even H^*, or that C′ was proven to behave like B in general. Advice injection and recursive coordination in other frameworks were likewise not measured. Considering a candidate improvement is not replaced by demonstrating one.

8 Validity and the limits of scope

8.1 A narrow subject and dependence on the specifications

This is a single language, a single repository, a single model identifier and a single CLI version, and the authoring and selection of specifications involved the author's judgement. Replicating seven specifications is not a replication across seven independent systems. In particular CT is a single specification, so we cannot say that another problem with the same graph shape would show the same cost difference. We also did not measure how often CT-like structures arise in practice.

8.2 A bundled intervention and unobservable state

Because session continuation and ordering constraints change together, the cost difference cannot be separated causally. Conversion by billing item and decomposition by task are possible, but they are not the effects of individual causes. The correspondence with the initial cache read is not a causal identification of the internal cache mechanism. Nonces do not guarantee that all sharing was blocked; fixed context and provider state remain. The CLI versions of the probes and of the main experiment also differ, so we do not transfer small-probe results to every boundary of the main experiment.

8.3 Order, operational changes and statistical assumptions

We verified the 49–49 balance of the leading policy, but arm order depends on the replication index and independence from time-varying behaviour is not guaranteed. There were changes A-5 and A-6, re-runs, and responses to usage limits. That sensitivity analyses do not change the aggregate conclusion does not mean every confound was removed. The sign-flip test and the bootstrap rely on assumptions of independence and exchangeability across pairs; if service state is correlated across adjacent replications, uncertainty may be underestimated.

8.4 Model approximation and the amount of calibration

Calibration used one replication per specification, so it cannot estimate the variance of exploration or history well. Defining exploration as everything before the first edit classifies later re-exploration as work. $\widehat{S}$ is the minimum observed context, not a measurement of the fixed system context itself. Output volume after continuation, tool behaviour and the accumulation of history are not predicted adequately. We ran no sensitivity analysis that shifts the exploration/work boundary, and we did not update the frozen prediction to fit these results.

8.5 The scope of the quality metric and few discrepancies

Acceptance tests are mechanical but do not represent every quality attribute. The 10-point margin is an operational judgement, not a criterion showing that quality is equal. Because binary discrepancies are few, even a corrected BCa faces small-sample and discreteness limits. The supplementary analysis using an exact binomial bound is likewise a check under the assumption of a common, independent probability of harm.

8.6 An asymmetry in reproduction caused by not redistributing the apparatus

Reproducibility is asymmetric between the analysis side and the experiment side. The measurements of all 196 runs are bundled, so re-running the aggregation, the decisions and the figures is self-contained within this package. The execution framework that ran the experiment and the repository under test, however, both lack a

license, and we decided not to redistribute them. What is published is a description of the procedure and the specifications, not the apparatus itself.

A third party therefore cannot independently re-acquire our numbers and verify them. What can be verified is whether the same conclusions follow from the bundled data. Applying the decision rules mechanically and preserving the pre-specification documents constrains analyst degrees of freedom; it is not a substitute for independent verification of data generation. This constraint is common to designs that measure an unmodifiable commercial API over a private repository, but being common does not remove it, so we confine our claims accordingly.

8.7 Absence of blinding, and evidence before publication

The designer of the study saw the direction of the pilot, and the idea for C′ came after observation. Calibrating from arm B alone avoids fitting the C/B ratio directly, but is not full blinding. Local Git timestamps are modifiable and do not by themselves prove an independent pre-registration time. We disclose the stored pre-specification documents and the amendment history; we do not state that an external registration or publication was completed.

9 Conclusion

Measuring a commercial coding-agent CLI and comparing 98 pairs, the criterion of a 20% cost reduction from conversation continuation was not met. Post-hoc analysis found medians of -7.9% and $+12.1$% even in the two W specifications where no continuation occurs, cautioning against reading the overall $+5.51$% as a general effect of continuation. That range is not an upper bound on noise, yet only CT exceeded it, with a median of $+50.6$% and a higher cost in all 14 pairs. Outside CT, C was cheaper in 44 of 84 pairs, and the increase in the overall mean was concentrated in CT's large differences.

Within CT the total turn count fell while cache writes and reads rose, and at fixed prices cache write was the largest item of the increase. Reporting the residual against the reported amount, together with the increase appearing in the leading task before any continuation, separates changes in billing items from a causal effect of inherited history. The overprediction across the five continued specifications is a concrete reason to re-examine the history model, the prices and the instrumentation.

What is needed next is an experiment that adds several CT specifications with different task content and compares continuation against its absence under the same execution order. This is required not only by the symmetry of the design but by an observation: the condition corresponding to the missing cell of Table 2 appeared in CT-A over 14 pairs, and the difference was not small (§6.7). The premise that the effect of serialization alone is negligible is inconsistent with that observation. Increasing the control specifications at a matched cost scale, and storing per-model cost and complete turn instrumentation, would allow the heterogeneity and the residual seen here to be verified independently. The present findings are limited to the specifications and the CLI observed, and establish neither a property of CT-shaped

work in general nor the superiority of any adaptive policy we did not evaluate.

A History of decisions and corrections

A.1 Corrections to the analysis

In v0.2 we corrected the input scale of H1 to the declared relative difference, and the statistic of H3 from a median to the mean corresponding to a difference in pass rates. The former H3 lower bound of 0.00 points reflected the concentration of a median when 97 of 98 pairs differ by zero; it was not an interval for a difference in pass rates. The earlier code and JSON are retained, re-run in a temporary area to confirm exact agreement, and the corrected results stored in separate code. Under H4b's earlier binary rule $(r > 1) \neq (\widehat{r} > 1)$ the mismatches are `n_db_chain` and `w_two_files`; under a three-valued increase/equal/decrease reading `s17_w_two_files` also mismatches. Neither satisfies the all-agree condition. The earlier CT-excluded $p \simeq 0.90$ was on the USD scale and differs from the relative-scale $p = 0.4950$ of the main text.

Item	Stored earlier result or statement	Treatment here
Scale of H1	$p = 0.0047$ and interval on USD differences	Corrected to the relative difference of the pre-analysis document: $p = 0.0013$, interval $[-3.18, +10.12]$%. The earlier USD median interval was $[-0.0173, +0.0581]$
Statistic of H3	Median lower bound of 0.00 points	Corrected to the mean pass-rate difference; lower bound -5.10 points. The criterion is still met
Direction under H4b	Binary decision including equality, described as "two mispredicted decreases"	The earlier code's mismatch set and the three-valued description are kept separate. Neither agrees throughout
Bias of the prediction	Overprediction in 7/7	Corrected to 6/7, consistent with the table and the computation
Statement about TTL	Asserted warm up to about 10.2 minutes	The stored classification of the 612.1 s and 916.9 s cases is ambiguous. The definite lifetime is withdrawn
Continuation after failure	Session always discarded	Withdrawn: the implementation carries no such blanket guarantee
Additional intervals	Not previously reported	Percentile intervals from within-specification resampling and a conservative binomial bound for acceptance are added as post-hoc analyses

Table 12: Corrections made at the integration of 4 September 2026. Neither the raw data nor the decision thresholds were changed.

The earlier version is stored under `paper/revision/original/` with a per-file SHA-256 recorded in `manifest.json`. The frozen results remain in `analysis/e2_results_`

`main.json` and the corrected results in `paper/revision/results.json`. That the principal conclusions did not change is not the same as there having been no error in the computation or the explanation. We correct the errors and preserve the reproducibility of the earlier results.

A.2 Arms that were planned but not run

Among the arms whose names survive in the pre-registration documents, we record those not run in the main experiment together with when and why they were set aside. Every one of these decisions predates the start of the main experiment (28 August 2026); none was dropped after seeing results.

Arm	Decided	Content and reason
A	2026-07-28	A single-agent, single continuous session reference point. Excluded at the scope freeze as outside the minimal configuration and kept as a candidate addition if budget allowed. Parked together with RQ5 (ablation), as recorded in §1 of the frozen protocol
D	2026-07-28	An adaptive scheduler. Parked at the same scope freeze; neither designed nor evaluated here
C′	2026-08-24	A budgeted lane. In a CT-only pilot ($n = 1$) the in-lane H moved 42.5k $\rightarrow$ 54.2k $\rightarrow$ 61.0k and both resumes cost more. Because H already reaches about 42.5k when the leading task ends, a rule that continues only while $H < H^*$ splits at the first boundary and C′ degenerates to B. Judged to carry no independent information and dropped by amendment A-2

Table 13: Arms not run. A and D were set aside at the scope freeze, C′ by an amendment before the main experiment.

The decisions on A and D predate not only the main experiment but the pilot (2 August 2026). The decision on C′ was a design judgement based on pilot observation and was recorded before the cost results of the main experiment were seen. Arm names begin at B because these labels are preserved as they appear in the pre-registration documents; correspondence with the stored documents was given priority over readability.

B Records of the mechanism measurements, and what remains undetermined

`probes/resume_probe_results.jsonl` stores both the initial attempts and the valid replications. The three M1 replications with a unique prefix are the init/resume pairs recorded around 02:09 on 31 July 2026 and are not mixed with the earlier attempts that used a shared prefix. In the three M4 pairs the cache write/read split agrees, while the total cost ratio including output reaches 1.082 in one case; we therefore report agreement of the cache split rather than exact agreement of total cost.

The long-gap probes are two cases at 612.1 and 916.9 seconds, both stored with the classification `ambiguous`. Cache write/read was 29,572/15,912 at 612.1 seconds and 29,417/15,912 at 916.9 seconds. These distinguish a state in which part of the

fixed context survives from full reuse of the history, and we do not treat them as an estimate of an exact retention time. There is also a read observed about 6.4 minutes after an initial call, but that is a continued measurement within the same session and includes the effect of lifetime refreshes. Satisfying the warm mechanism check does not mean passing all the long-gap and switch conditions of H4a.

`m6_results.jsonl` and `m7_results.jsonl` support the observations on added amount and initial cache read. We do not treat the twelve M7 measurements and the initial-state examples including M6 as one sample of the same size in a single table. Internal causes such as the provider's placement of breakpoints cannot be settled from these logs alone.

C Definitions of the additional analyses

The stratified bootstrap over the fixed seven specifications resamples 14 pairs with replacement from each specification and computes the median relative difference of the resulting 98 pairs. We repeat this 10,000 times and use linearly interpolated 2.5 and 97.5 percentiles. Because the specifications themselves were not sampled at random, this is distinct from population inference over a larger set of specifications.

For acceptance, writing $q = P(B \text{ succeeds}, C \text{ fails})$ and $g = P(B \text{ fails}, C \text{ succeeds})$, the pass-rate difference satisfies $g - q \geq -q$. The observed adverse discrepancy is 1/98, so with the one-sided 90% exact binomial upper bound U_q [3], $-U_q$ is a conservative lower bound on the pass-rate difference. Solving $\sum_{j=0}^{1} \binom{98}{j} U_q^j (1-U_q)^{98-j} = 0.10$ gives a lower bound of -3.91 points. This is a supplementary check assuming a common adverse probability and independence across pairs, and it does not guarantee per-specification acceptance quality.

C.1 v0.3 additional analyses and instrumentation reconciliation

The structural controls and per-specification intervals use the existing BCa function with 10,000 resamples and seed 20260802. The exact sign test uses $\min\{1, 2\sum_{j=0}^{\min(n_+,n_-)} \binom{n}{j} 2^{-n}\}$ over the n pairs remaining after removing ties, of which there were none. It is a supplementary test against a null model of equal probabilities of sign for independent pairs and does not claim randomization of arm order. The 70 continued pairs are also recorded in the stored JSON, with a median relative difference of $+6.90$% and a BCa 95% interval of $[-1.42, +12.17]$%.

`calls.jsonl` is an append-only history, and the earlier executor writes the usage of the final process to `run.json`. In 195 of the 196 runs the five aggregates (four token counts and cost) of a terminal suffix of the call history match the final usage. Seven runs carry extra leading history, and `m_mixed/B/rep5` did not match under a simple suffix sum. That run is not excluded from the primary cost or from any token aggregate; we use its `run.json`. All 28 CT runs match on all five items across all three calls each, and the per-task decomposition is confined to that verified scope.

Auxiliary-model use was checked in every run. Of the 518 call records, all 434 in fresh sessions carry an auxiliary model name in addition to the representative model, and none of the 84 in continued sessions do. There is no exception in any of the seven specifications; within CT the split is 42/42 for B and 14/42 for C. De-

tailed per-model `modelUsage` figures were not stored for the main experiment, so the auxiliary model's contribution cannot be isolated per run, and descriptions that compare per-turn quantities between arms therefore include both the amount of history and the model composition. The probes, however, stored the raw result including `modelUsage`. From 27 fresh-side records we read a cost of 0.012116 USD per auxiliary call, 12,043 input tokens, 15 output tokens, and zero cache read and write; varying the payload from 300 to 16,000 characters moves the input by 27 tokens, but only because every probe used the same 500-line prefix and so held the prompt size fixed; changing the prefix moves the auxiliary input proportionally (§6.7). In all 27 records the four `usage` items equal the main model's own figures, so the auxiliary model's tokens stay out of that aggregate while its cost is added to `total_cost_usd`; the auxiliary model therefore sits inside the residual ϵ of §6.6. Its behaviour can be characterized through this route (§6.7), but per-call prompt sizes were not stored, so its per-call cost in the main experiment is not determined, and whether the work substitutes for the main model or is incidental to starting a session likewise remains undecidable from these logs.

Paired differences within CT were computed by matching the B-side and C-side calls of the same replication index. Peak context is the median over 14 replications in each arm. The comparison of inherited amounts uses the peak context from calibration (arm B, one replication), which does not match the replication count of the main experiment, so the ratios are treated as an approximate contrast. Ratios of reported cost to the fixed-price conversion were computed per specification and per arm, and the per-item composition of the residual was not recovered. On the B side of CT, the cache-read total over turn rows exceeds the result aggregate by 632,352 tokens and the write total by 26,734; the C side agrees. `output_tokens_partial` is an undetermined value at the start of a turn and is not used in cost aggregation. The CLI's internal aggregation scope, auxiliary processing and pricing structure cannot be recovered uniquely from this storage format, so we do not attribute the residual solely to a difference in prices. This is the limit on separating changes in billing items, per-model billing, and the cost of inherited history alone.

D Reproduction and artifacts

The canonical Japanese manuscript is `paper/main.tex` and this English manuscript is `paper/main_en.tex`; both draw their numbers, tables and figures from the same dataset and correction JSON. The generating script is `paper/render_assets.py`, and the additional results are stored in `paper/revision/v0.3/results.json`. Tables for this manuscript are the English variants under `paper/generated/en/`, derived from the same numbers. The steps to rebuild the manuscript and its figures are:

```
python3 analysis/revision_audit.py
python3 analysis/supplement_v03.py
.venv-figs/bin/python paper/render_assets.py
bash paper/build.sh en
```

The analysis corrections use only the Python standard library, the figures use NumPy and Matplotlib, and PDF generation uses Tectonic, which is XeLaTeX-compatible. The build script uses a fetched Tectonic or an executable on PATH; TeX resources

not yet fetched require network access on the first run. `paper/BUILD.md` records the environment and the role of each file.

The analysis can be re-run from the stored data alone, but re-running the experiment requires access to the commercial service, the execution framework and the fixed version of the repository under test. We do not claim that the sources of this manuscript alone allow the 196 runs to be re-acquired. The limits this creates are stated in §8.6.

References

[1] ccusage contributors. Cost modes. ccusage documentation, 2026. Accessed 2026-09-05. https://ccusage.com/guide/cost-modes.

[2] ccusage contributors. Session usage. ccusage documentation, 2026. Accessed 2026-09-05. https://ccusage.com/guide/session-reports.

[3] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika, 26(4):404–413, 1934. https://doi.org/10.1093/biomet/26.4.404.

[4] Bradley Efron and Robert J. Tibshirani. An Introduction to the Bootstrap. Chapman & Hall, 1993. https://doi.org/10.1007/978-1-4899-4541-9.

[5] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. arXiv:2405.19888, version 1, 2024. https://arxiv.org/abs/2405.19888v1.

[6] llm-d contributors. Precise prefix cache routing. llm-d documentation, 2026. Accessed 2026-09-05. https://llm-d.ai/docs/well-lit-paths/foundations/precise-prefix-cache-routing.

[7] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don't break the cache: An evaluation of prompt caching for long-horizon agentic tasks. arXiv:2601.06007, version 2, 2026. https://arxiv.org/abs/2601.06007v2.

[8] Belinda Phipson and Gordon K. Smyth. Permutation p-values should never be zero: Calculating exact p-values when permutations are randomly drawn. Statistical Applications in Genetics and Molecular Biology, 9(1):Article 39, 2010. https://doi.org/10.2202/1544-6115.1585.

[9] Rakibul Hasan Rajib, Mengxin Zheng, and Qian Lou. AgentServeSim: A hardware-aware simulator for multi-turn LLM agent serving. arXiv:2606.09613, version 1, 2026. https://arxiv.org/abs/2606.09613v1.

[10] Ray contributors. Prefix-aware routing. Ray documentation, 2026. Accessed 2026-09-05. https://docs.ray.io/en/latest/serve/llm/user-guides/prefix-aware-routing.html.

[11] Cosmo Santoni. Contextual memory virtualisation: DAG-based state management and structurally lossless trimming for LLM agents. arXiv:2602.22402, version 1, 2026. https://arxiv.org/abs/2602.22402v1.

[12] taekim34. Session resume (–continue) invalidates entire prompt cache, causes massive rate limit consumption. anthropics/claude-code, GitHub issue 42338, 2026-04-02, 2026. Practitioner report; accessed 2026-09-05. https://github.com/anthropics/claude-code/issues/42338.

[13] Michelle Vaccaro. Preregistration for experiments with AI agents. arXiv:2606.11217, version 1, 2026. https://arxiv.org/abs/2606.11217v1.

[14] Xu Yang, Lunyiu Nie, Ethan Chandra, Stanislav Gannutin, Fangru Lin, and Swarat Chaudhuri. When parallelism pays off: Cohesion-aware task partitioning for multi-agent coding. arXiv:2606.00953, version 1, 2026. https://arxiv.org/abs/2606.00953v1.

[15] Ying Yuan, Pengfei Zuo, Bo Wang, Zhangyu Chen, Zhipeng Tan, and Zhou Yu. DualMap: Enabling both cache affinity and load balancing for distributed LLM serving. arXiv:2602.06502, version 1, 2026. https://arxiv.org/abs/2602.06502v1.

[16] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. arXiv:2312.07104, version 2, 2024. https://arxiv.org/abs/2312.07104v2.